

Karta – 1

Kompleksowy Poradnik: Implementacja Rejestracji Twórców i Zarządzania Portfelami w TipJar

Wprowadzenie

Niniejszy poradnik stanowi szczegółowy przewodnik krok po kroku dotyczący projektowania i implementacji systemu rejestracji oraz logowania dla twórców na platformie TipJar. Obejmuje on integrację z popularnymi dostawcami tożsamości (logowanie społecznościowe), opcjonalne logowanie Web3 (Sign-In with Ethereum - SIWE), a także kluczowy element – automatyczne tworzenie i powiązanie dewelopersko kontrolowanego portfela Circle dla każdego twórcy. Celem jest zapewnienie bezpiecznego, wydajnego, skalowalnego i przede wszystkim przyjaznego dla użytkownika procesu, który jednocześnie wykorzystuje nowoczesne standardy Web2 i Web3.

Część A: Logowanie Społecznościowe (Google, Twitch, TikTok, YouTube) i Web3 (SIWE)

A.1 Koncept i Cel

- **Cel Główny:** Umożliwienie twórcom jak najprostszego i najszybszego dołączenia do TipJar. Logowanie przez istniejące konta społecznościowe (Google, Twitch itp.) znacząco redukuje tarcie związane z koniecznością tworzenia i zapamiętywania nowych danych logowania. Zwiększa to współczynnik konwersji przy rejestracji i buduje zaufanie poprzez wykorzystanie znanych i zaufanych dostawców tożsamości.
- **Logowanie Web3 (SIWE - Sign-In with Ethereum):** Jako opcjonalna ścieżka dla twórców zaznajomionych z Web3, SIWE pozwala na uwierzytelnienie za pomocą ich portfela kryptowalutowego (np. MetaMask). Jest to zgodne z etosem decentralizacji i daje użytkownikom Web3-native poczucie kontroli nad swoją tożsamością. Dodanie tej opcji zwiększa atrakcyjność platformy w ekosystemie kryptowalut.
- **Wyzwania:**
 - Różnice w implementacji API i zakresach danych zwracanych przez poszczególnych dostawców OAuth (np. Google vs. Twitch).
 - Potencjalne ograniczenia w dostępie do API niektórych platform (np. TikTok, YouTube) dla celów czysto uwierzytelniających; może być konieczne dokładne zbadanie ich aktualnych polityk i możliwości.
 - Konieczność bezpiecznego zarządzania wieloma kluczami **Client ID** i **Client Secret**.
 - Obsługa błędów, odrzuceń autoryzacji przez użytkownika oraz odwoływania dostępu.

- **Kwestie Regulacyjne (GDPR/CCPA):** Pozyskiwanie danych profilowych od dostawców społecznościowych (nawet podstawowych jak email czy nazwa) podlega regulacjom o ochronie danych osobowych. **Kluczowe jest skonsultowanie się z prawnikiem** w celu zapewnienia zgodności z GDPR, CCPA i innymi lokalnymi przepisami, w tym opracowanie przejrzystej polityki prywatności i mechanizmów uzyskiwania zgody użytkownika.

A.2 Dobór Narzędzi i Programów

1. **Standard Uwierzytelniania (Logowanie Społecznościowe): OAuth 2.0** (oraz **OpenID Connect (OIDC)** jako warstwa nad OAuth 2.0, szczególnie dla Google).
 - *Dlaczego?* Standard branżowy, bezpieczny, szeroko wspierany. Umożliwia delegowaną autoryzację bez konieczności zarządzania hasłami użytkowników.
 - *Alternatywy:* SAML (bardziej korporacyjny), własne systemy (niezalecane).
2. **Standard Uwierzytelniania (Logowanie Web3): EIP-4361 (Sign-In with Ethereum - SIWE).**
 - *Dlaczego?* Standard dla uwierzytelniania użytkowników Ethereum za pomocą podpisania wiadomości portfelem. Zapewnia bezpieczne i zdecentralizowane logowanie.
3. **Framework Backendowy i Biblioteki (NestJS/Node.js):**
 - **Passport.js:** Middleware do uwierzytelniania, niezwykle elastyczne, z setkami "strategii" dla różnych dostawców.
 - *Strategie:* `passport-google-oauth20`, `passport-twitch-new` (lub inna aktualna). Dla TikToka/YouTube może być potrzebny research lub bezpośrednia implementacja klienta OAuth 2.0.
 - *Zalety:* Modularność, abstrakcja złożoności OAuth, duża społeczność.
 - **Biblioteki dla SIWE (Backend):** `ethers.js` lub `viem` do weryfikacji podpisów i odzyskiwania adresu z wiadomości SIWE.
 - **Zarządzanie Sesjami/Tokenami: JSON Web Tokens (JWT).**
 - *Dlaczego?* Bezstanowe (stateless), dobre dla skalowalnych API. Backend generuje podpisany JWT po pomyślnym uwierzytelnieniu.
 - *Alternatywy:* Sesje oparte na ciasteczkach (stateful), PASETO.
4. **Biblioteki Frontendowe (Next.js/React):**
 - Standardowe `Workspace` API lub `axios` do komunikacji z backendem TipJar.
 - Dla SIWE: `viem` lub `ethers.js` do interakcji z portfelem przeglądarkowym użytkownika (np. MetaMask) w celu podpisania wiadomości SIWE.
5. **Narzędzia Monitorowania: Datadog, New Relic, Sentry** lub podobne.
 - *Dlaczego?* Do śledzenia wydajności i błędów własnego API oraz czasów odpowiedzi i błędów zewnętrznych dostawców OAuth i API Circle.

A.3 Techniczny Opis Implementacji (Krok po Kroku)

Opiszemy szczegółowo przepływ dla Google OAuth z NestJS, Passport.js i JWT, a następnie wskażemy adaptacje dla SIWE.

Przepływ Google OAuth 2.0 (z NestJS, Passport.js):

1. Konfiguracja u Dostawcy (Google Cloud Console):

- Utwórz projekt w Google Cloud Console.
- W sekcji "APIs & Services" -> "Credentials", utwórz "OAuth client ID".
- Wybierz typ "Web application".
- Podaj autoryzowane **Redirect URIs** (np. `http://localhost:3001/auth/google/callback` dla dewelopmentu i `https://api.twojadomena.com/auth/google/callback` dla produkcji). *Ważne: `localhost:3000` z Twojego przykładu to port frontendu, callback powinien iść do backendu, np. `localhost:3001` lub na ten sam port jeśli używasz proxy.*
- Zapisz uzyskane **Client ID** i **Client Secret**. Przechowuj je bezpiecznie jako zmienne środowiskowe w pliku `.env` backendu.
- Upewnij się, że API "Google People API" jest włączone dla projektu, aby móc pobierać dane profilu.

2. Konfiguracja NestJS (Moduł **AuthModule**, Strategia **GoogleStrategy**, Kontroler **AuthController**):

google.strategy.ts:

```
// google.strategy.ts
import { PassportStrategy } from '@nestjs/passport';
import { Strategy, VerifyCallback, Profile } from 'passport-google-oauth20';
import { Injectable, HttpException, HttpStatus, Logger } from '@nestjs/common'; // Dodano
Logger
import { AuthService } from './auth.service'; // Serwis do logiki autoryzacji i użytkowników

@Injectable()
export class GoogleStrategy extends PassportStrategy(Strategy, 'google') {
  private readonly logger = new Logger(GoogleStrategy.name); // Inicjalizacja Loggera

  constructor(private authService: AuthService) {
    super({
      clientID: process.env.GOOGLE_CLIENT_ID, // Z .env
      clientSecret: process.env.GOOGLE_CLIENT_SECRET, // Z .env
      callbackURL: process.env.GOOGLE_CALLBACK_URL, // Z .env np.
        '<http://localhost:3001/auth/google/callback>'
      scope: ['email', 'profile'],
    });
  }

  async validate(accessToken: string, refreshToken: string, profile: Profile, done:
    VerifyCallback): Promise<any> {
    const { id: googleId, name, emails, photos } = profile;
    const primaryEmail = emails?.[0]?.value;
```

```

    const displayName = name?.givenName ? `${name.givenName} ${name.familyName ||
    ""}`.trim() : profile.displayName;
    const avatarUrl = photos?.[0]?.value;

    if (!googleId || !primaryEmail) {
      this.logger.error('Failed to get primary data from Google profile.');
```

return done(new HttpException('Nie udało się uzyskać podstawowych danych z Google',
HttpStatus.UNAUTHORIZED), false);

```

    }

    try {
      const user = await this.authService.validateOAuthUser(
        'google',
        googleId,
        primaryEmail,
        displayName,
        avatarUrl,
        accessToken,
        refreshToken
      );
      done(null, user);
    } catch (error) {
      this.logger.error(`Error in GoogleStrategy validate for googleId ${googleId}:
      ${error.message}`, error.stack);
      done(new HttpException('Błąd podczas przetwarzania danych użytkownika',
      HttpStatus.INTERNAL_SERVER_ERROR), false);
    }
  }
}

```

○

auth.module.ts:

```

// auth.module.ts
import { Module } from '@nestjs/common';
import { PassportModule } from '@nestjs/passport';
import { JwtModule } from '@nestjs/jwt';
import { AuthService } from './auth.service';
import { GoogleStrategy } from './google.strategy';
import { AuthController } from './auth.controller';
// import { UsersModule } from '../users/users.module'; // Jeśli UsersService jest w osobnym
module
// import { CircleModule } from '../circle/circle.module'; // Jeśli CircleService jest potrzebny w
AuthService

@Module({

```

```

imports: [
  // UsersModule,
  // CircleModule,
  PassportModule.register({ defaultStrategy: 'google' }),
  JwtModule.register({
    secret: process.env.JWT_SECRET,
    signOptions: { expiresIn: process.env.JWT_EXPIRES_IN || '1h' },
  }),
],
providers: [AuthService, GoogleStrategy /*, JwtStrategy */],
controllers: [AuthController],
// exports: [AuthService],
})
export class AuthModule {}

```

○

auth.controller.ts:

```

// auth.controller.ts
import { Controller, Get, UseGuards, Req, Res, HttpException, HttpStatus, Logger } from
 '@nestjs/common'; // Dodano Logger
import { AuthGuard } from '@nestjs/passport';
import { AuthService } from './auth.service';
import { Request, Response } from 'express';

@Controller('auth')
export class AuthController {
  private readonly logger = new Logger(AuthController.name); // Inicjalizacja Loggera

  constructor(private authService: AuthService) {}

  @Get('google')
  @UseGuards(AuthGuard('google'))
  async googleAuth(@Req() req: Request) {
    this.logger.log('Initiating Google OAuth flow.');
```

```

  }

  @Get('google/callback')
  @UseGuards(AuthGuard('google'))
  async googleAuthRedirect(@Req() req: Request, @Res() res: Response) {
    this.logger.log('Received callback from Google.');
```

```

    if (!req.user) {
      this.logger.error('Google authentication failed, no user object in request.');
```

```

      throw new HttpException('Uwierzytelnianie Google nie powiodło się',
        HttpStatus.UNAUTHORIZED);
    }
  }
}

```

```

const tipJarUser = req.user as { userId: string; email: string; /* inne pola */ };
this.logger.log(`User ${tipJarUser.email} authenticated via Google. Generating JWT.`);

const jwtTokenObject = await this.authService.login(tipJarUser);

res.cookie('jwt', jwtTokenObject.accessToken, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'lax',
  maxAge: parseInt(process.env.JWT_EXPIRES_IN_SECONDS || '3600') * 1000,
});
this.logger.log(`JWT cookie set for ${tipJarUser.email}. Redirecting to frontend
dashboard.`);
return res.redirect(`${process.env.FRONTEND_URL}/dashboard`);
}
}

```

○

3. Frontend (Next.js/React):

- Użytkownik klika przycisk "Zaloguj przez Google".
- Aplikacja frontendowa wykonuje `window.location.href = '<http://localhost:3001/auth/google>'`; (lub odpowiedni URL produkcyjny backendu).
- Po pomyślnym zalogowaniu i przekierowaniu zwrotnym (zgodnie z logiką w `AuthController`), frontend (np. strona `/dashboard`) powinien sprawdzić obecność ciasteczka `jwt` (lub innego mechanizmu przekazania tokenu) i zapisać go/używać do dalszych zapytań.
- Przy każdym kolejnym zapytaniu do chronionych endpointów API TipJar, frontend dołącza JWT do nagłówka `Authorization: Bearer <token>`.

4. Obsługa Błędów Sieciowych (w strategii lub serwisie):

Podczas wywołań API do Google (choć Passport.js to w dużej mierze obsługuje) lub później do API Circle, kluczowa jest obsługa błędów.

```

// Przykład bardziej generycznej obsługi błędów z axios (jeśli byłby używany bezpośrednio)
// import axios from 'axios';
// try {
//   const response = await
axios.get("<https://api.externalservice.com/data>")("<https://api.externalservice.com/data>"),
//   { headers: { Authorization: `Bearer ${accessToken}` } }
// };
// } catch (error) {
//   if (axios.isAxiosError(error)) {

```

```
// this.logger.error('Błąd Axios:', error.response?.data || error.message);
// if (error.response?.status === 401) {
//   throw new HttpException('Błąd autoryzacji z usługą zewnętrzną',
//     HttpStatus.UNAUTHORIZED);
// } else if (error.response?.status === 429) {
//   throw new HttpException('Przekroczono limit żądań do usługi zewnętrznej',
//     HttpStatus.TOO_MANY_REQUESTS);
// }
// throw new HttpException('Błąd sieciowy podczas komunikacji z usługą zewnętrzną',
//   HttpStatus.BAD_GATEWAY);
// }
// this.logger.error('Nieznany błąd sieciowy:', error);
// throw new HttpException('Wystąpił nieoczekiwany błąd sieciowy',
//   HttpStatus.INTERNAL_SERVER_ERROR);
// }
```

-
- Zaleca się użycie bibliotek typu `axios-retry` lub implementację własnej logiki ponawiania prób z *exponential backoff* dla operacji, które mogą chwilowo zawieść.

Implementacja SIWE (Sign-In with Ethereum):

1. Frontend:

- Użytkownik klika "Zaloguj Portfelem".
- Frontend (używając `viem` lub `ethers.js`):
 1. Pobiera `nonce` (losowy ciąg znaków zapobiegający replay attacks) z backendu (`GET /auth/siwe/nonce`).
 2. Prosi użytkownika o połączenie portfela (np. MetaMask), pobiera adres użytkownika.
 3. Tworzy wiadomość zgodną ze standardem EIP-4361, zawierającą m.in.: domenę aplikacji, adres użytkownika, oświadczenie, URI, wersję, chain ID, nonce, datę wystawienia.
 4. Prosi użytkownika o podpisanie tej wiadomości za pomocą jego portfela.
 5. Wysyła oryginalną wiadomość i podpis (`signature`) do backendu (`POST /auth/siwe/verify`).

2. Backend (NestJS):

- Implementacja `SiweStrategy` (analogicznie do `GoogleStrategy` lub jako osobny serwis).
- Endpoint `/auth/siwe/nonce`: Generuje i zwraca kryptograficznie bezpieczny `nonce`, tymczasowo przechowując go (np. w sesji lub krótkotrwałym cache) w powiązaniu z sesją użytkownika/IP.
- Endpoint `/auth/siwe/verify`:
 1. Odbiera wiadomość i podpis.

2. Weryfikuje podpis (używając `ethers.js` lub `viem`), odzyskując adres z podpisu.
3. Porównuje odzyskany adres z adresem w wiadomości.
4. Weryfikuje `nonce` (czy zgadza się z wcześniej wygenerowanym i czy nie został już użyty).
5. Weryfikuje inne pola wiadomości (domena, chain ID).
6. Jeśli weryfikacja pomyślna:
 - Wywołuje `authService.validateSiweUser(recoveredAddress, messageFields)`.
 - `AuthService` sprawdza, czy użytkownik z tym adresem portfela istnieje. Jeśli nie, tworzy go i inicjuje tworzenie portfela Circle.
 - Generuje JWT i zwraca go do frontendu.

A.4 Zabezpieczenia (Rozwinięcie)

- **CSRF Protection:**
 - **OAuth 2.0 state token:** Passport.js (przy prawidłowej konfiguracji) automatycznie obsługuje parametr `state` w przepływie OAuth.
 - **JWT i CSRF:** Przy używaniu ciasteczek `HttpOnly` dla JWT, wszystkie żądania modyfikujące stan (POST, PUT, DELETE) muszą być chronione przed CSRF (np. przez Double Submit Cookie pattern lub synchronizer token pattern). NestJS wspiera `csurf` dla aplikacji opartych na sesjach; dla API bezstanowych (JWT) można to zaimplementować manualnie lub użyć dedykowanych bibliotek.
- **Bezpieczne Przechowywanie Sekretów:**
 - Wszystkie sekrety (`GOOGLE_CLIENT_SECRET`, `JWT_SECRET`, `CIRCLE_API_KEY`, `CIRCLE_ENTITY_SECRET`, klucze do szyfrowania bazy danych) w zmiennych środowiskowych (`.env` lokalnie, menedżery sekretów na produkcji - AWS Secrets Manager, Google Secret Manager, HashiCorp Vault).
- **HTTPS:** TLS 1.3 na produkcji dla całej komunikacji.
- **Walidacja Tokenów (JWT):** Weryfikacja sygnatury, `exp`, `iss` (issuer), `aud` (audience). `@nestjs/jwt` z `JwtStrategy` to obsługuje.
- **Minimalny Zakres Uprawnień (OAuth Scope):** `['email', 'profile']` to dobry start.
- **Bezpieczne Przekierowania (Open Redirect):** Walidacja URLi przekierowań. Nie konstruuje URLi z danych od użytkownika bez ścisłej walidacji na białej liście domen.
- **Rate Limiting:** Użyj `nestjs-throttler` do ograniczenia liczby prób logowania / żądań do endpointów z jednego IP.
- **Ochrona przed XSS:**
 - **JWT Storage:** Ciasteczka `HttpOnly` (`Secure`, `SameSite=Lax/Strict`) są bezpieczniejsze niż `localStorage`.

- **Sanityzacja Danych:** Dane wyświetlane na frontendzie muszą być sanityzowane/escapowane (np. `sanitize-html` lub wbudowane mechanizmy Reacta).
- **Audyty API Dostawców:** Śledzenie dokumentacji API Google, Circle pod kątem zmian w bezpieczeństwie i zaleceń.

A.5 Adresowanie Wymagań Niefunkcjonalnych (Rozwinięcie)

- **Bezpieczeństwo:** Implementacja A.4, regularne przeglądy kodu, testy penetracyjne.
- **Wydajność (np. < 500 ms callback, 95% żądań < 1s dla 1000 użytkowników):**
 - Asynchroniczność Node.js/NestJS, optymalizacja zapytań DB (indeksy), caching (Redis).
 - Testy obciążeniowe (k6, Locust) do weryfikacji.
- **Skalowalność (Horizontal scaling):** Bezstanowe JWT, skalowalny backend (instancje za load balancerem), skalowalna baza danych (PostgreSQL z replikami).
- **Użyteczność:** Logowanie 1-kliknięciem, jasne komunikaty błędów.
- **Niezawodność (Uptime 99,9%):** Redundancja (multi-AZ), obsługa błędów i retry dla API zewnętrznych (`axios-retry`), monitoring (Prometheus, Grafana, Sentry).
- **Utrzymywalność:** Modułowość NestJS, TypeScript, dobre praktyki, dokumentacja (Swagger/OpenAPI, JSDoc/TSDoc), testy.
- **Kompatybilność:** Elastyczność Passport.js, responsywny frontend.

Część B: Powiązanie z Developer-Controlled Circle Wallet (Rozwinięcie)

B.1 Koncept i Cel (Rozwinięcie)

Portfele kontrolowane przez dewelopera (DCW) w Circle API są kluczowe dla TipJar, umożliwiając automatyczne tworzenie portfeli USDC dla twórców bez obciążania ich technicznymi aspektami zarządzania kluczami.

- **Zalety dla TipJar:**
 - **Płynny Onboarding:** Portfel tworzony automatycznie w tle.
 - **Kontrola Serwera:** TipJar inicjuje transakcje (wyплаты, agregacja opłat).
 - **Sponsorowanie Gazu:** Możliwość pokrywania opłat przez Gas Station.
 - **Uprozczone Odzyskiwanie:** Odzyskanie dostępu do konta TipJar = dostęp do środków.
- **Ryzyko Centralizacji i Rozważania:**
 - **Bezpieczeństwo Sekretów:** Wyciek `CIRCLE_API_KEY` lub `CIRCLE_ENTITY_SECRET` jest katastrofalny.
 - **Zależność od Circle:** Awaria Circle = problemy z portfelami.
 - **Plan Awaryjny:** Warstwa abstrakcji (`CircleService`), backupy mapowań ID.
- **Alternatywy (Account Abstraction - ERC-4337):**
 - ERC-4337 (portfele smart kontraktowe) z Paymasterami to bardziej zdecentralizowana opcja dla przyszłej ewolucji, oferująca social recovery i

gas sponsoring przy zachowaniu kontroli przez użytkownika. Dla MVP, DCW jest prostsze.

B.2 Dobór Narzędzi i Programów (Rozwinięcie)

1. **Circle Programmable Wallets API:** Podstawowy interfejs REST API.
2. **Circle SDK dla Node.js: @circle-fin/developer-controlled-wallets** (npm).
 - o *Dlaczego?* Upraszcza interakcję, enkapsuluje logikę HTTP, obsługę błędów, autoryzację, zapewnia typowanie.
 - o *Instalacja:* `npm install @circle-fin/developer-controlled-wallets`.
3. **Baza Danych (PostgreSQL + Prisma):** Schemat użytkownika TipJar (*User*):
 - o `id` (String, PK, UUID)
 - o `email` (String, unikalny)
 - o `googleId` (String, opcjonalny, unikalny, indeksowany)
 - o `twitchId` (String, opcjonalny, unikalny, indeksowany)
 - o `walletAddressSIWE` (String, opcjonalny, unikalny, indeksowany)
 - o `circleWalletId` (String, opcjonalny, unikalny, indeksowany) - ID portfela Circle.
 - o `mainWalletAddress` (String, opcjonalny, indeksowany) - Adres blockchain portfela Circle na domyślnej sieci.
 - o `isCircleSetupComplete` (Boolean, domyślnie `false`).
 - o ... (inne pola profilu)

B.3 Techniczny Opis Implementacji (Rozwinięcie z @circle-fin/developer-controlled-wallets)

Krok 0: Inicjalizacja Klienta Circle SDK (w CircleService NestJS) Kod jak w poprzedniej odpowiedzi (sekcja B.3 `circle.service.ts` - inicjalizacja `circleClient` w `onModuleInit`). Upewnij się, że `CIRCLE_WALLET_SET_ID` jest również w zmiennych środowiskowych i używany.

Krok 1: Wywołanie Tworzenia Portfela W `AuthService`, po pomyślnym utworzeniu nowego użytkownika TipJar (niezależnie od metody logowania - OAuth czy SIWE), wywołaj `this.circleService.provisionUserWallet(newTipJarUser.id, newTipJarUser.email)`. Rozważ wywołanie asynchroniczne (np. przez event lub dodanie zadania do kolejki), aby nie blokować odpowiedzi dla użytkownika.

Krok 2: Implementacja Metody `provisionUserWallet` w `CircleService` Kod jak w poprzedniej odpowiedzi (sekcja B.3, metoda `provisionUserWallet` w `circle.service.ts`). Kluczowe elementy:

- Użycie `this.circleClient.createWallets(...)` z `walletSetId`,
`blockchains: [process.env.DEFAULT_BLOCKCHAIN || 'MATIC-AMOY']`,

`count: 1, accountType: 'SCA'` (dla kompatybilności z Gas Station na EVM) i odpowiednimi metadanymi.

- Użycie **klucza idempotencji** (`X-Request-Idempotency-Key` - SDK powinno to obsługiwać, ale warto to potwierdzić; jeśli nie, generuj UUID i przekazuj jako opcję żądania).
- Zapisanie zwróconych `circleWalletId` i `address` (`mainWalletAddress`) do rekordu użytkownika TipJar w bazie.
- Solidna obsługa błędów zwracanych przez SDK/API Circle (logowanie, rzucanie odpowiednich `HttpException`).

Krok 3: Obsługa Błędów Asynchronicznych (jeśli tworzenie portfela jest w tle)

- **Kolejka Zadań:** BullMQ (z Redisem) lub RabbitMQ. Zadanie zawiera `tipJarUserId` i `email`.
- **Worker:** Przetwarza zadania, wywołując `provisionUserWallet`.
- **Strategie Retry:** Automatyczne ponawianie prób z exponential backoff.
- **Dead Letter Queue (DLQ):** Dla zadań, które permanentnie zawiodły.
- **Powiadomienia:** Alerty o błędach w DLQ (np. Sentry, Slack).
- **Stan Użytkownika:** W UI, status "Konfiguracja portfela w toku..." lub błędu.

B.4 Zabezpieczenia (Rozwinięcie)

- **Ochrona Klucza API Circle i Sekretu Encji:** Jak w A.4 (menedżery sekretów).
- **Kontrola Dostępu do Endpointów Backendu:** Endpointy wywołujące `CircleService` muszą być chronione.
- **Walidacja Danych:** Walidacja parametrów przekazywanych do `CircleService` i odpowiedzi z SDK.
- **Audit Logging:** Logowanie wszystkich operacji na portfelach Circle (kto, co, kiedy, wynik, ID operacji, ID portfela).
- **Ochrona przed Nadużyciami (Rate Limiting):** Na endpointy inicjujące operacje Circle.
- **Szyfrowanie Danych Wrażliwych w Bazie TipJar:** `circleWalletId`, `mainWalletAddress` mogą być uznane za wrażliwe. Rozważ szyfrowanie w spoczynku (AES-256) z bezpiecznym zarządzaniem kluczami szyfrującymi.

B.5 Adresowanie Wymagań Niefunkcjonalnych (Rozwinięcie)

- **Bezpieczeństwo:** Bezpieczeństwo MPC Circle + praktyki z B.4.
- **Wydajność (np. tworzenie portfela < 2s; 99% żądań < 3s):** Asynchroniczne tworzenie portfela, monitoring metryk.
- **Skalowalność:** Skalowalność API Circle + skalowalny backend TipJar.
- **Użyteczność:** Automatyczne, niewidoczne dla użytkownika tworzenie portfela.
- **Niezawodność (Uptime 99,95% Circle + redundancja TipJar):** Idempotencja (SDK!), retry, DLQ, monitoring Circle.
- **Utrzymywalność:** Dedykowany `CircleModule` i `CircleService`, testy z mockowaniem SDK.

- **Kompatybilność:** Circle DCW wspiera główne sieci EVM i Solana. Wybierz `DEFAULT_BLOCKCHAIN` w `.env`.
-

Ogólna Architektura Bezpieczeństwa i Najlepsze Praktyki (Rozwinięcie)

- **Defense in Depth:** WAF (Cloudflare, AWS WAF), firewall, szyfrowanie TLS 1.3, walidacja wejścia/wyjścia, bezpieczne kodowanie (OWASP Top 10), szyfrowanie danych w spoczynku, zasada minimalnych uprawnień, MFA dla adminów.
 - **HTTPS Everywhere:** Wymuszenie TLS 1.3.
 - **Input Validation:** `class-validator` w NestJS.
 - **Rate Limiting & Throttling:** `nestjs-throttler` (np. 100 żądań/min na IP dla logowania/rejestracji).
 - **Web Application Firewall (WAF):** Konfiguracja pod kątem typowych ataków.
 - **Dependency Scanning:** Dependabot (GitHub), Snyk, `npm audit`.
 - **Regularne Audyty Bezpieczeństwa:** Co kwartał testy penetracyjne i przeglądy kodu.
 - **Zasada Minimalnych Uprawnień:** Dla użytkowników, kluczy API.
 - **Blockchain-based Audit:** Dla kluczowych operacji (utworzenie portfela), hash logu zapisywany on-chain (np. Polygon) jako dowód. TxHash zapisywany w logach TipJar.
-

Ulepszenia i Poprawki (Rozwinięcie)

- **Asynchroniczne Tworzenie Portfela:** BullMQ + Redis + Workery NestJS.
 - **Lepsza Obsługa Błędów:** Integracja z Sentry, specyficzne kody błędów, alerty.
 - **Monitorowanie:** Prometheus + Grafana (metryki API TipJar, Circle, kolejek, zasobów).
 - **Odświeżanie Tokenów JWT:** Implementacja Refresh Tokenów (dłuższy czas życia, przechowywane bezpiecznie np. HttpOnly cookie) do odnawiania Access Tokenów (krótki czas życia).
 - **Plan Testów Integracyjnych:**
 - **Jednostkowe (Jest):** Mockowanie `passport-google-oauth20`, `@circle-fin/developer-controlled-wallets`, `PrismaClient`.
 - **Integracyjne (Jest + `testcontainers`):** Testowanie serwisów z prawdziwą (testową) bazą danych.
 - **E2E API (Jest + `supertest`):** Testowanie pełnych przepływów API z mockowaniem zewnętrznych usług (Google, Circle API) za pomocą `nock` lub `msw`.
 - **Testy Obciążeniowe (k6, Locust):** np. 5000 jednoczesnych użytkowników na endpointach logowania/rejestracji.
 - **Testy Bezpieczeństwa:** OWASP ZAP, zewnętrzne testy penetracyjne.
-

Zakończenie

Ten rozbudowany poradnik dostarcza kompleksowego przeglądu implementacji systemu rejestracji i zarządzania portfelami dla twórców w TipJar. Wdrożenie przedstawionych rozwiązań, w tym wykorzystanie oficjalnego SDK Circle dla Node.js, powinno zapewnić solidne fundamenty pod bezpieczną, wydajną i skalowalną platformę. Pamiętaj, że jest to baza, którą należy dostosować do specyficznych potrzeb i dalszej ewolucji projektu.

Lista Niezbędnych Narzędzi i Technologii

Frontend (np. Web - Next.js/React, Mobile - React Native/Flutter/Swift/Kotlin):

- **Framework/Biblioteka UI:** Next.js (React) lub odpowiednik dla mobile.
- **Język:** TypeScript.
- **Styling:** Tailwind CSS (lub odpowiednik).
- **Zarządzanie Stanem:** Zustand, Redux Toolkit (lub odpowiednik mobilny).
- **Interakcja z API:** Axios lub wbudowany `Workspace`.
- **Interakcja z Portfelami Web3 (dla SIWE/MetaMask):** `viem`, `ethers.js`.
- **Circle Modular Wallets Web SDK (@circle-fin/modular-wallets-core):** Dla funkcji Passkey lub interakcji z modular wallets od strony frontendu (opcjonalnie, jeśli takie funkcje są planowane).

Backend (NestJS/Node.js):

- **Framework:** NestJS.
- **Język:** TypeScript.
- **Środowisko Uruchomieniowe:** Node.js (zalecana wersja LTS).
- **Uwierzytelnianie OAuth:** Passport.js z odpowiednimi strategiami (np. `passport-google-oauth20`, `passport-twitch-new`).
- **Uwierzytelnianie SIWE:** `ethers.js` lub `viem` (do weryfikacji podpisów).
- **Tokeny JWT:** `@nestjs/jwt`.
- **Integracja z Circle:** Oficjalne SDK `@circle-fin/developer-controlled-wallets`.
- **ORM/Interakcja z Bazą Danych:** Prisma.
- **Walidacja Danych:** `class-validator`, `class-transformer`.
- **Obsługa HTTP:** Wbudowany `HttpModule` w NestJS (oparty na Axios), lub bezpośrednio `axios` / `axios-retry`.
- **Rate Limiting:** `nestjs-throttler`.
- **Kolejki Zadań (Opcjonalnie):** BullMQ (z Redisem) lub RabbitMQ.

Baza Danych:

- PostgreSQL (zalecane) lub MySQL / MariaDB.

Infrastruktura i DevOps:

- **Konteneryzacja:** Docker, Docker Compose.
- **Hosting Frontend:** Vercel, Netlify, AWS Amplify, Firebase Hosting, GitHub Pages.

- **Hosting Backend i Bazy Danych:** AWS (EC2, ECS, Fargate, RDS, Lambda), Google Cloud (Compute Engine, Cloud Run, Kubernetes Engine, Cloud SQL), Microsoft Azure (App Service, AKS, Azure SQL), Render, Heroku, DigitalOcean.
- **CI/CD (Integracja i Dostarczanie Ciągłe):** GitHub Actions, GitLab CI, Jenkins, CircleCI, Bitbucket Pipelines.
- **API Gateway/Reverse Proxy:** AWS API Gateway, Nginx, Traefik, Kong.
- **Menedżer Sekretów:** AWS Secrets Manager, Google Secret Manager, HashiCorp Vault, Doppler.

Bezpieczeństwo:

- **Web Application Firewall (WAF):** Cloudflare WAF, AWS WAF, ModSecurity.
- **Skanowanie Podatności Zależności:** Dependabot (GitHub), Snyk, `npm audit`, `yarn audit`.
- **Skanowanie Kodu (SAST/DAST):** SonarQube (SAST), OWASP ZAP (DAST), Burp Suite (manualne testy).
- **Biblioteki do Sanityzacji HTML (Frontend):** `sanitize-html`, DOMPurify.

Monitorowanie, Logowanie i Alertowanie:

- **Logowanie Aplikacji:** Wbudowany Logger NestJS, Winston, Pino; wysyłanie logów do centralnego systemu (np. ELK Stack, Grafana Loki, Datadog Logs).
- **Zbieranie Metryk:** Prometheus (z `prom-client` w Node.js).
- **Wizualizacja Metryk i Dashboardy:** Grafana.
- **Śledzenie Błędów i APM (Application Performance Monitoring):** Sentry, Datadog APM, New Relic APM.
- **Status Page (dla zależności):** Oficjalne strony statusowe Circle, Google Cloud itp.

Testowanie:

- **Framework Testowy (Backend):** Jest (domyślny z NestJS).
- **Testy E2E API (Backend):** `supertest` (z Jest).
- **Mockowanie HTTP (Backend):** `nock`, `msw` (Mock Service Worker).
- **Testy Jednostkowe/Integracyjne (Frontend):** Jest, React Testing Library, Vitest.
- **Testy E2E (Frontend):** Playwright, Cypress, Puppeteer.
- **Testy Obciążeniowe:** k6, Locust, Artillery.
- **Kontenery dla Testów:** `testcontainers-node` (do uruchamiania np. bazy danych w Dockerze na potrzeby testów).

Narzędzia Deweloperskie i Współpraca:

- **IDE:** VS Code (z wtyczkami dla TypeScript, NestJS, Prisma, Docker, ESLint, Prettier), WebStorm.
- **Formatowanie Kodu:** Prettier.
- **Linting Kodu:** ESLint (z odpowiednimi konfiguracjami dla TypeScript i NestJS).
- **Kontrola Wersji:** Git.
- **Repozytoria Kodu:** GitHub, GitLab, Bitbucket.
- **Zarządzanie Projektem/Zadaniami:** Jira, Trello, Asana, Linear, Notion.

- **Komunikacja Zespołowa:** Slack, Microsoft Teams, Discord.
 - **Projektowanie UI/UX:** Figma, Sketch, Adobe XD, Penpot.
 - **Dokumentacja API:** Swagger/OpenAPI (NestJS ma wbudowane wsparcie poprzez [@nestjs/swagger](#)).
 - **Wirtualizacja/Kontenery (Lokalny Development):** Docker Desktop.
-